

DOCMIND

Agentic Multimodal Enterprise Document Intelligence System

Project Type: B.Tech Minor Project

Domain: Artificial Intelligence, Generative AI, Multimodal RAG, Agentic AI

Team: Team Nexus

1. Abstract

DocMind is an intelligent enterprise document understanding and question-answering system that combines **Multimodal Retrieval-Augmented Generation (RAG)** with an **agentic AI architecture**.

Organizations maintain large collections of technical manuals, reports, contracts, Standard Operating Procedures (SOPs), presentations, scanned documents, tables, diagrams, and images. Conventional keyword-based search systems are inefficient for such repositories because they have limited contextual understanding and cannot effectively reason over multimodal information.

DocMind addresses these challenges through a structured document-processing and agentic retrieval pipeline. **Docling** is used for document parsing, while **ONNX Runtime with PaddleOCR** handles OCR requirements. **Qwen2.5-VL** provides visual understanding of images, diagrams, and other visual content. Text and visual representations are indexed using **BGE-M3** and **SigLIP2** respectively, with **Qdrant** serving as the primary vector database.

A **LangGraph-based multi-agent workflow** coordinates query planning, retrieval, reranking, reasoning, verification, and response generation. The language-model layer uses a fallback chain consisting of **GPT-OSS 120B** → **Nemotron 3 Nano 30B-A3B** → **DeepSeek V4-Flash** → **Qwen 27B** → **Llama 3.3 70B Instant**.

The system provides users with concise, context-aware answers accompanied by lightweight source citations through a modern **React + Framer Motion** interface.

2. Problem Statement

Enterprise knowledge is distributed across large numbers of heterogeneous documents. Finding relevant information manually can require considerable time and effort.

Traditional search systems primarily depend on keyword matching and therefore have difficulty answering semantic and multi-step questions. Conventional RAG systems improve retrieval but can still suffer from irrelevant context, hallucinated responses, insufficient source attribution, and limited understanding of tables, diagrams, and images.

The proposed system aims to overcome these limitations by developing an intelligent document platform capable of understanding multimodal content, dynamically planning retrieval, reasoning over multiple sources, verifying generated information, and producing grounded responses.

3. Motivation

The increasing volume and complexity of enterprise documents makes knowledge retrieval slow and inefficient. Existing systems often treat documents primarily as text and lack the ability to understand images, tables, diagrams, and scanned content.

DocMind aims to transform static document repositories into an intelligent knowledge system by combining multimodal RAG with specialised AI agents. The system is designed to make document-based information retrieval faster, more accurate, contextual, and explainable.

4. Objectives

The objectives of DocMind are:

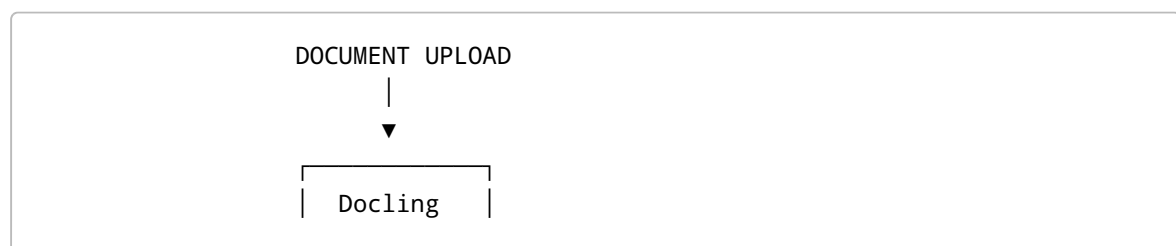
1. To develop a scalable multimodal document intelligence platform.
 2. To process textual, tabular, visual, and scanned document content.
 3. To implement a Retrieval-Augmented Generation pipeline for grounded question answering.
 4. To use an agentic workflow for query planning, retrieval, reasoning, verification, and response generation.
 5. To implement semantic and hybrid retrieval with reranking.
 6. To provide concise answers with relevant source citations.
 7. To support queries requiring information from multiple documents.
 8. To evaluate retrieval and generation quality using appropriate RAG evaluation metrics.
 9. To develop an intuitive web interface for document upload and conversational interaction.
-

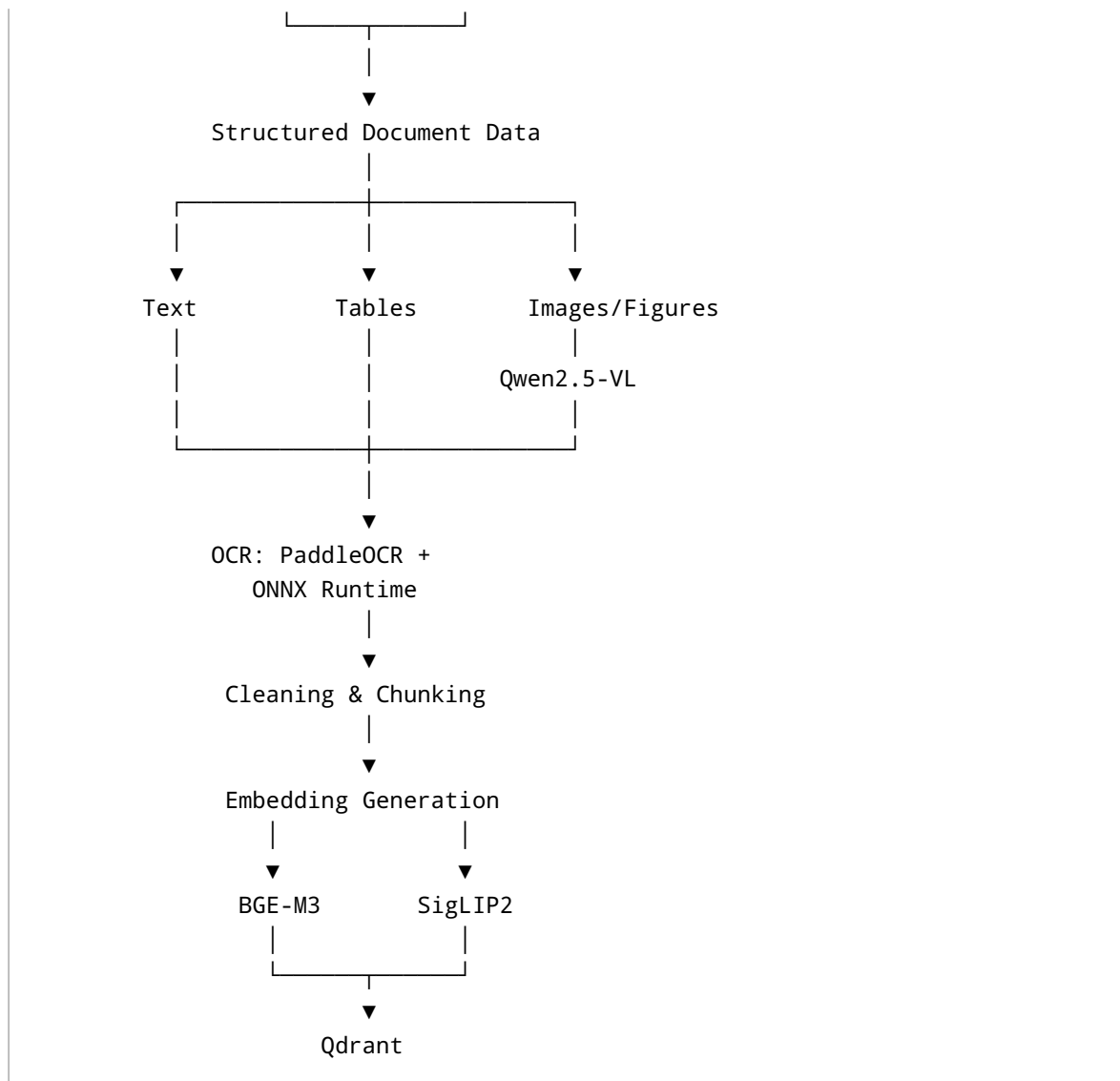
5. Proposed System

DocMind consists of two interconnected pipelines:

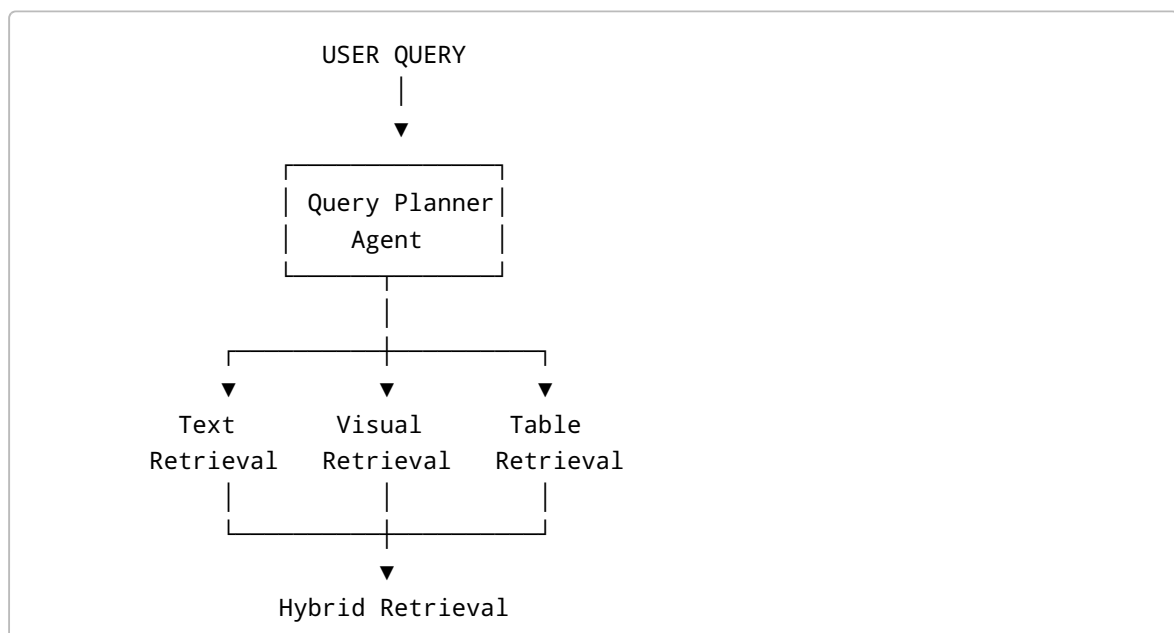
A. Document Ingestion Pipeline

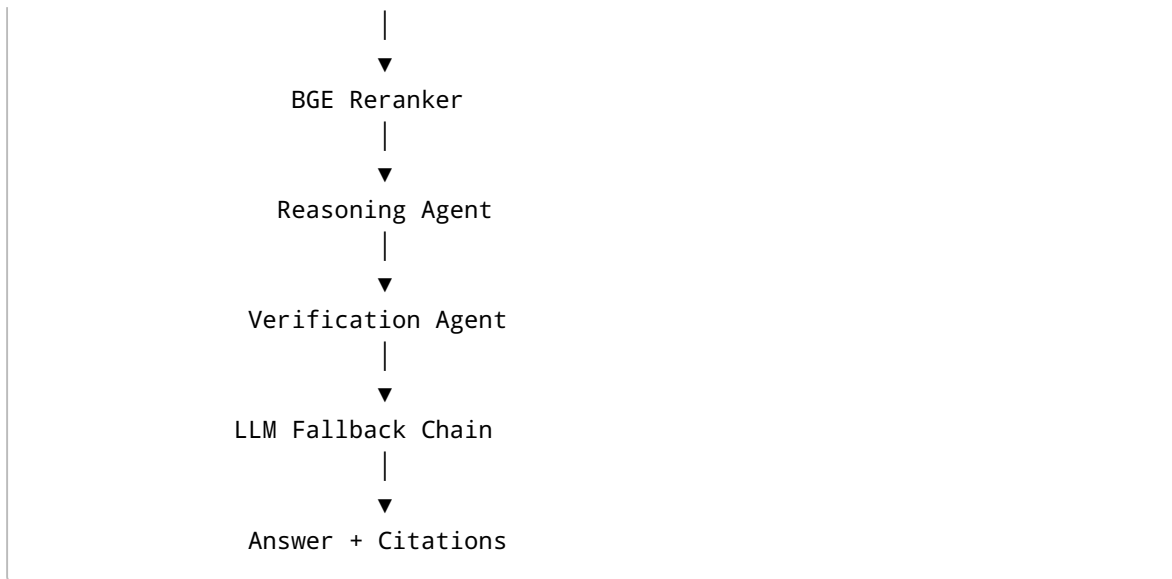
Raw documents are parsed, analysed, structured, embedded, and indexed before they become available for querying.





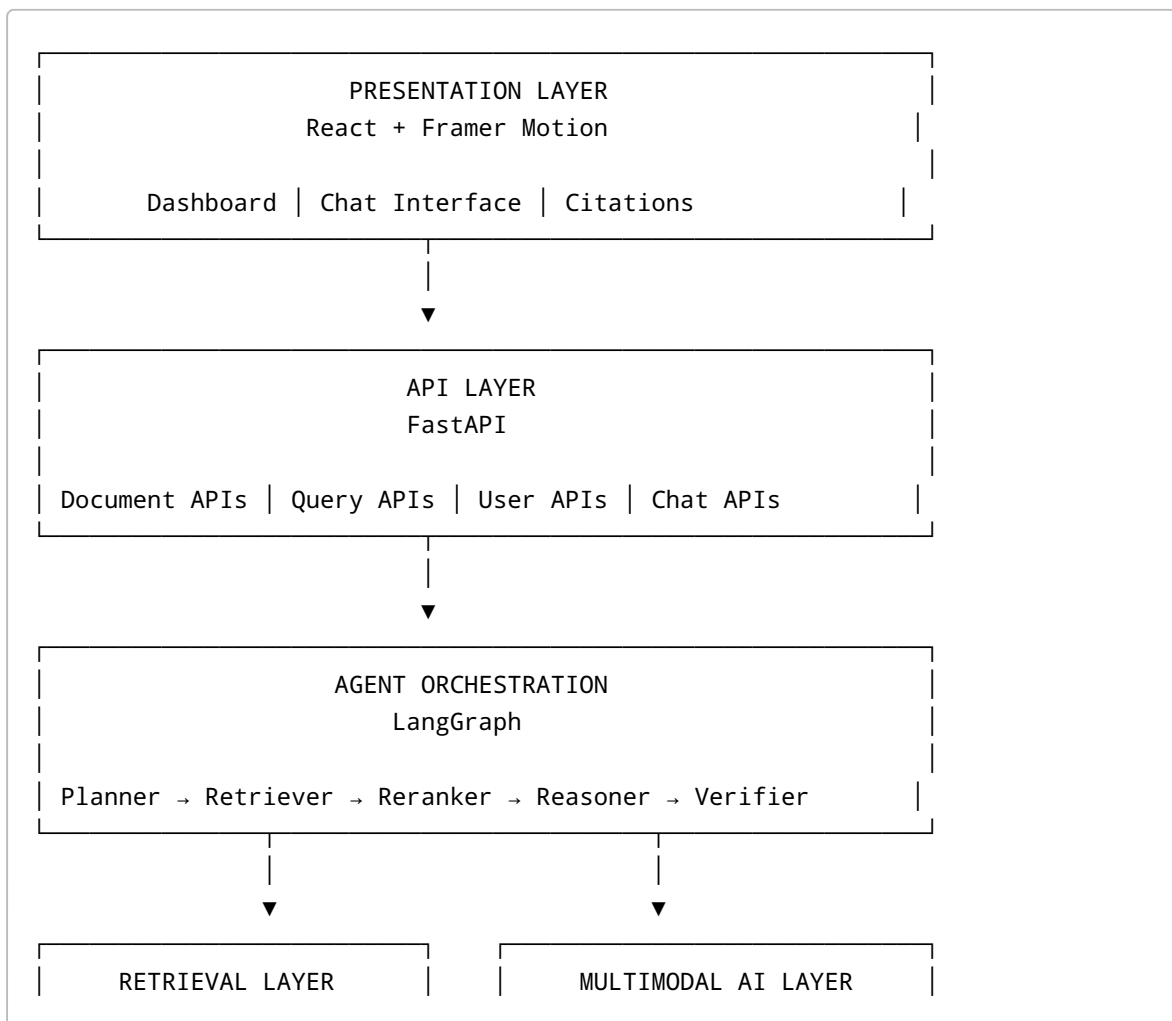
B. Query Processing Pipeline

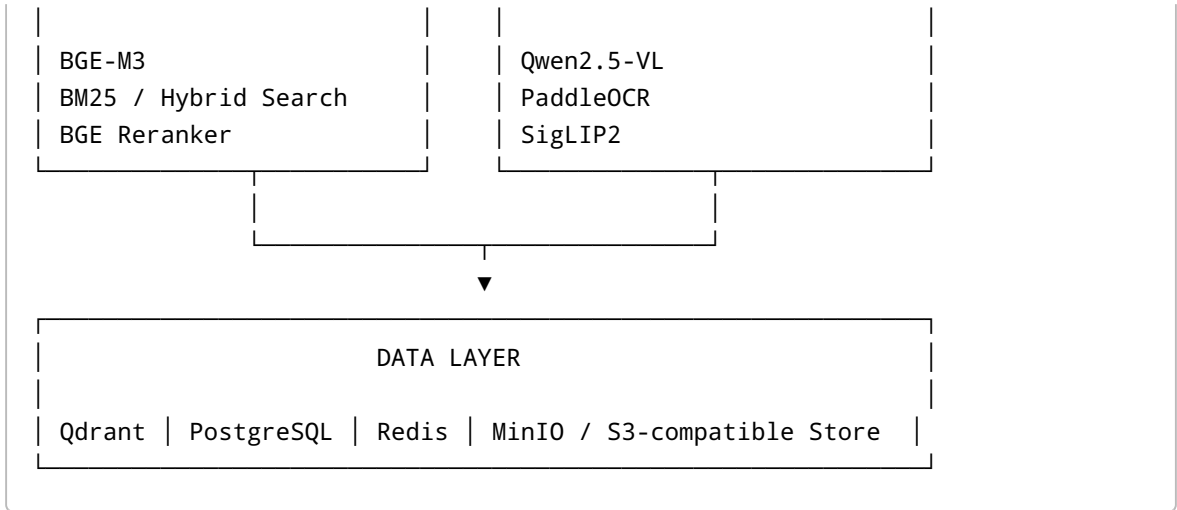




6. High-Level Architecture

The system is divided into presentation, API, orchestration, retrieval, multimodal AI, and data layers.





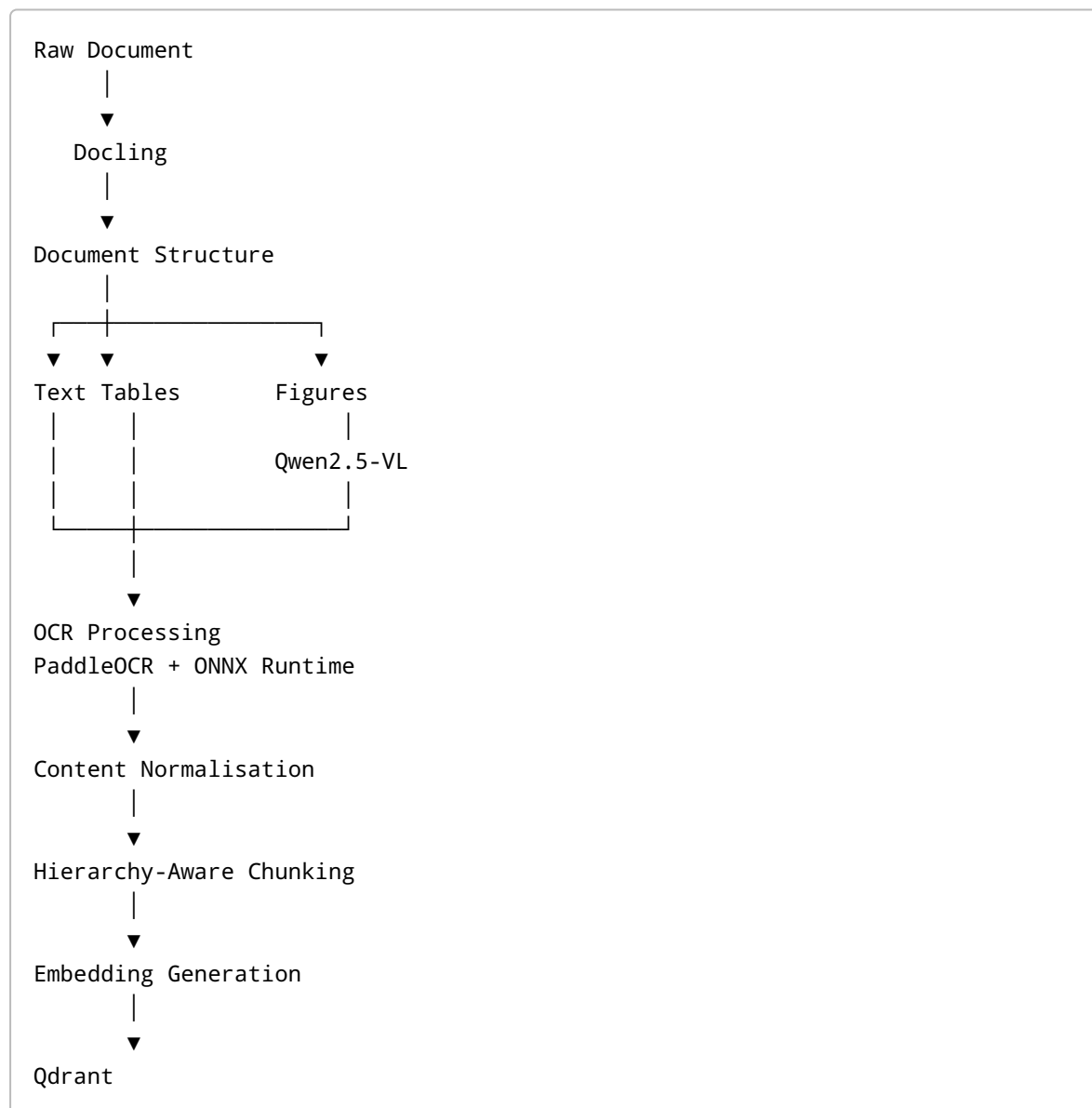
7. Technology Stack

Layer	Technology	Purpose
Frontend	React	Web application interface
UI Animation	Framer Motion	Smooth interactions and animations
Backend	FastAPI	REST API and backend services
Agent Framework	LangGraph	Multi-agent orchestration
Document Parser	Docling	Structured document parsing
OCR	PaddleOCR	OCR for scanned/image content
OCR Runtime	ONNX Runtime	Efficient OCR model execution
Vision Model	Qwen2.5-VL	Visual document understanding
Text Embeddings	BGE-M3	Semantic text representations
Visual Embeddings	SigLIP2	Visual representations
Reranker	BGE Reranker	Retrieval relevance optimisation
Vector Database	Qdrant	Vector search and metadata filtering
Relational Database	PostgreSQL	Users, documents, chats, metadata
Cache	Redis	Caching and temporary state
Object Storage	MinIO / S3	Original documents and media
Primary LLM	GPT-OSS 120B	Reasoning and generation
LLM Fallback 1	Nemotron 3 Nano 30B-A3B	Reasoning and agentic tasks
LLM Fallback 2	DeepSeek V4-Flash	Long-context reasoning and generation
LLM Fallback 3	Qwen 27B	General reasoning and generation

Layer	Technology	Purpose
LLM Fallback 4	Llama 3.3 70B Instant	Final fallback
Containers	Docker	Deployment and reproducibility
Version Control	Git/GitHub	Source-code management

8. Document Ingestion Pipeline

The ingestion pipeline transforms raw documents into structured and searchable representations.



The ingestion process retains document structure and metadata so that retrieved information can later be mapped back to the original source.

9. Document Parsing with Docling

Docling serves as the primary document parsing framework.

The parser is responsible for extracting and preserving structural elements such as:

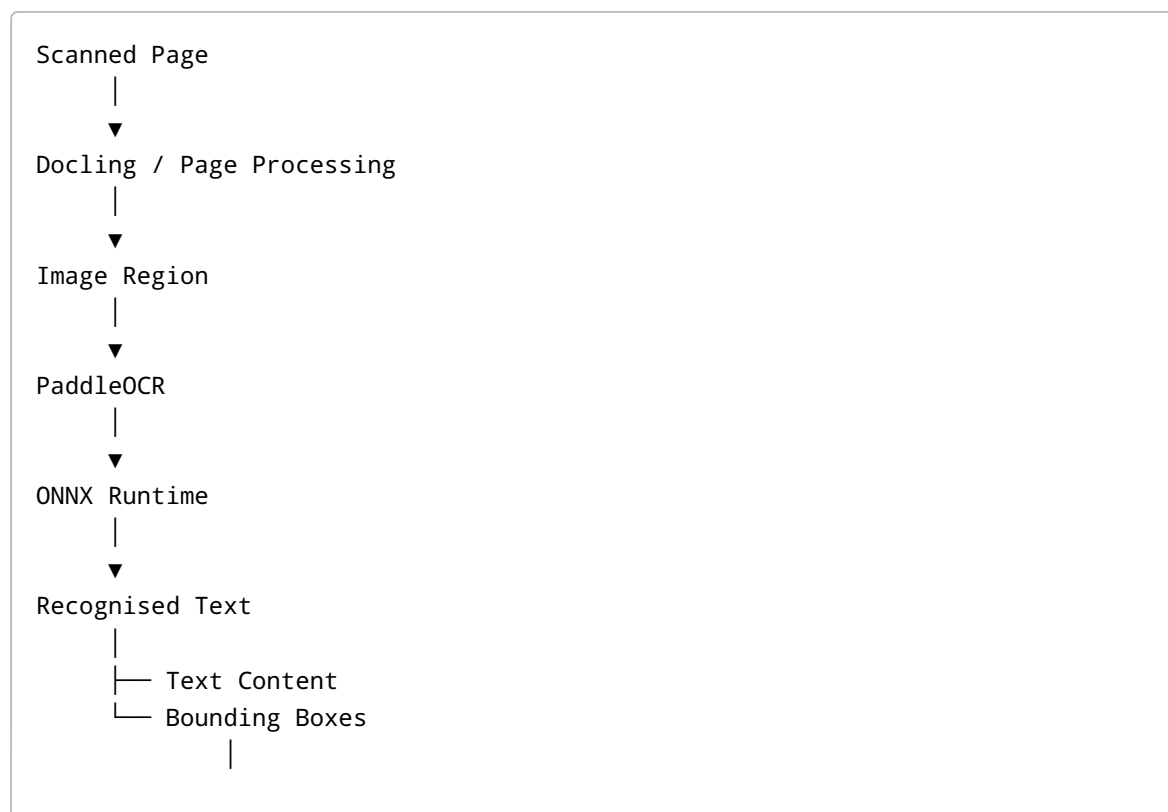
- paragraphs;
- headings;
- sections;
- lists;
- tables;
- figures;
- images;
- page information;
- document hierarchy; and
- positional information.

Maintaining this structure is important because the relationship between a heading, its text, its tables, and associated figures can contain valuable context.

10. OCR Architecture

Scanned pages and image-based documents require Optical Character Recognition.

DocMind uses **PaddleOCR executed through ONNX Runtime** as part of its OCR pipeline.

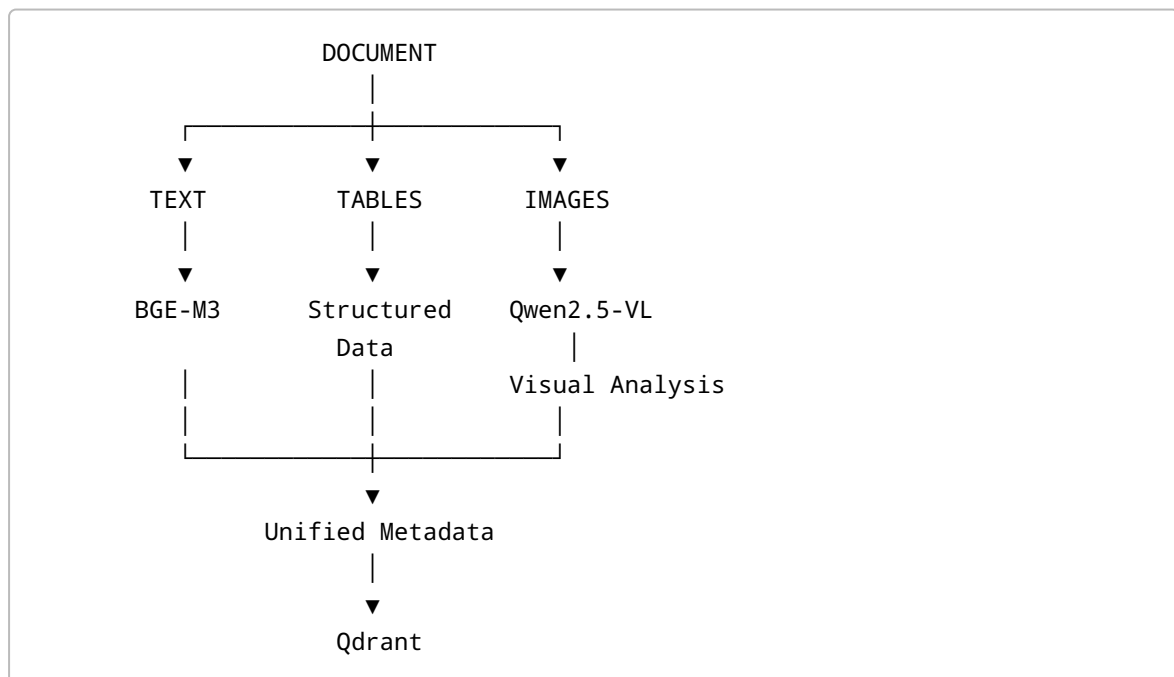


Structured Metadata

Bounding-box information allows extracted text to retain its approximate position within the source page.

11. Multimodal Document Understanding

DocMind separates document elements according to their modality.



This architecture allows a query to retrieve different forms of evidence when necessary.

12. Vision Processing with Qwen2.5-VL

Qwen2.5-VL is the dedicated vision-language model of DocMind.

It can be used for:

- diagram understanding;
- chart interpretation;
- image analysis;
- figure description;
- visual question answering;
- extraction of semantic information from visual content.

For example, if a document contains a technical diagram, Qwen2.5-VL can analyse the diagram and produce information that can subsequently assist retrieval and answer generation.

The vision model remains separate from the main LLM fallback chain.

13. Embedding Architecture

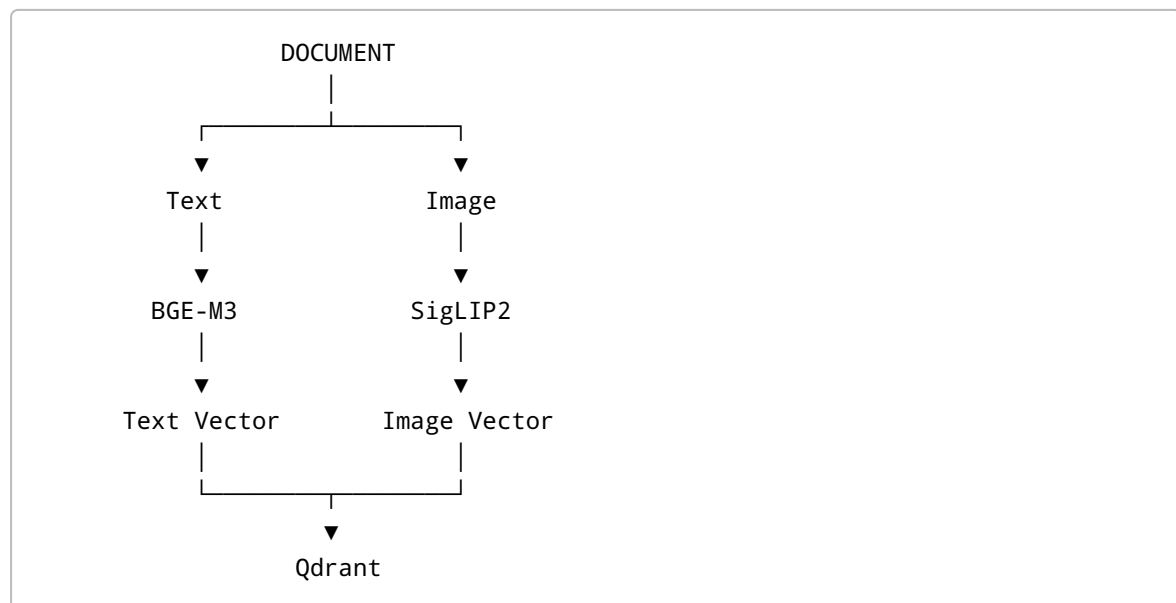
DocMind uses separate representations for textual and visual information.

Text

BGE-M3 generates semantic embeddings for text chunks.

Visual Content

SigLIP2 generates embeddings for visual elements.



Metadata associated with each vector can include:

```
document_id
page_number
section
heading
chunk_id
content_type
source_location
bounding_box
parent_section
```

14. Hierarchy-Aware Chunking

Instead of splitting documents blindly according to a fixed character count, DocMind uses document structure to create contextually meaningful chunks.

Example:



Each chunk retains its relationship with the parent document and section.

This improves retrieval quality and allows citations to identify the original page and section.

15. Qdrant Vector Database

Qdrant is used as the primary vector database.

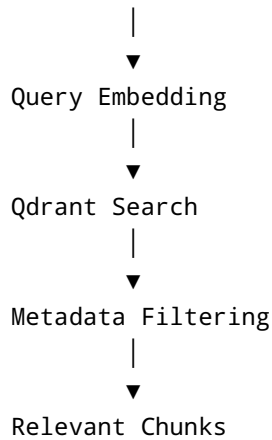
It stores:

- text embeddings;
- visual embeddings;
- chunk identifiers;
- document identifiers;
- page information;
- content type;
- section metadata;
- source references.

The system can combine similarity search with metadata filtering.

For example:

Query:
"Find the safety procedure in the 2025 manual."

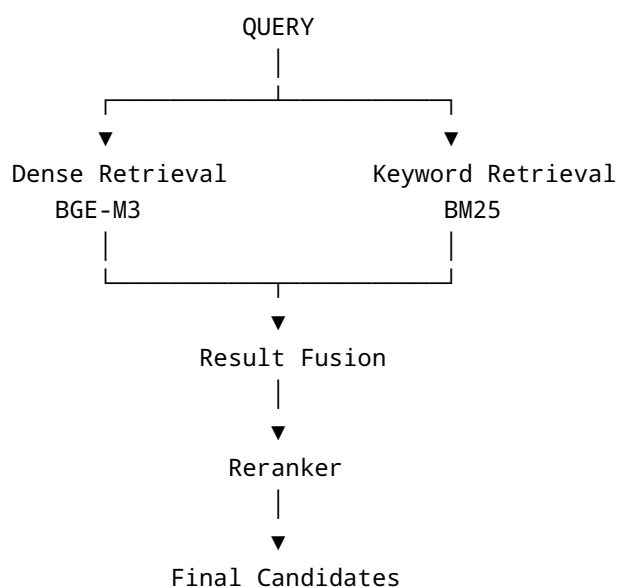


16. Hybrid Retrieval

DocMind will use a hybrid retrieval strategy rather than relying entirely on vector similarity.

This is particularly useful for technical information such as:

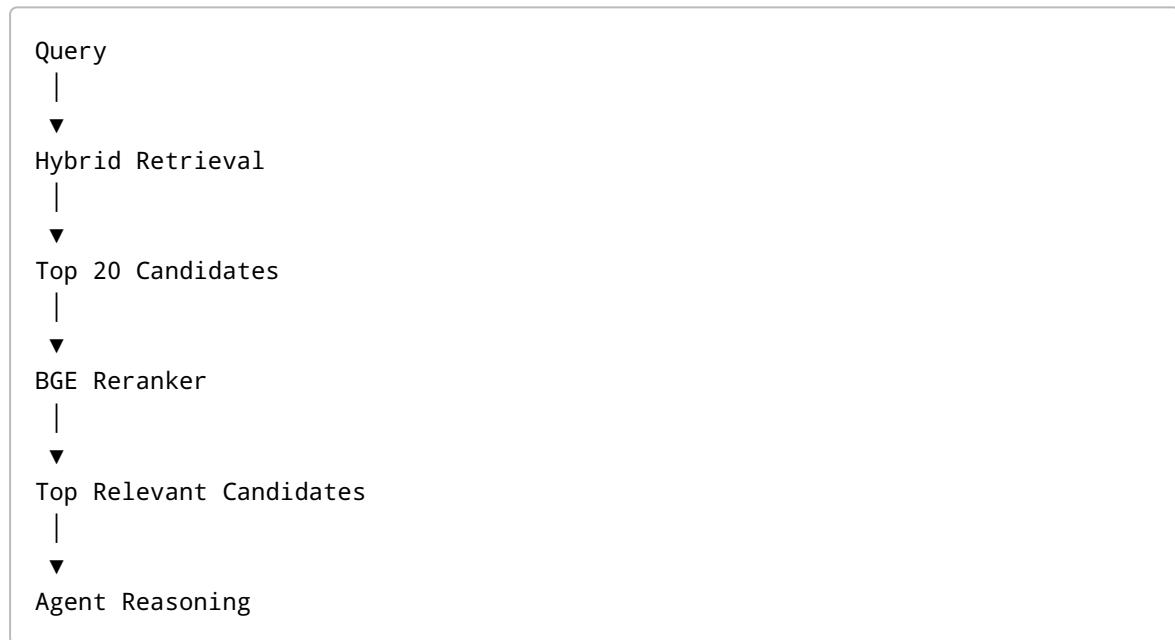
- product numbers;
- model numbers;
- part codes;
- abbreviations;
- exact terminology.



17. Reranking

Initial retrieval may produce several potentially relevant results. A reranker is therefore used to select the strongest candidates.

BGE Reranker evaluates the relevance between the query and retrieved content.



This reduces irrelevant context entering the language model.

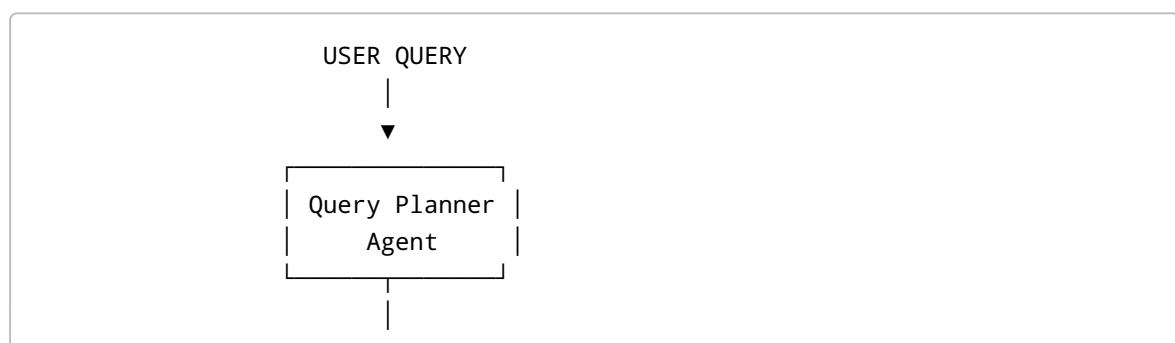
18. Agentic Architecture

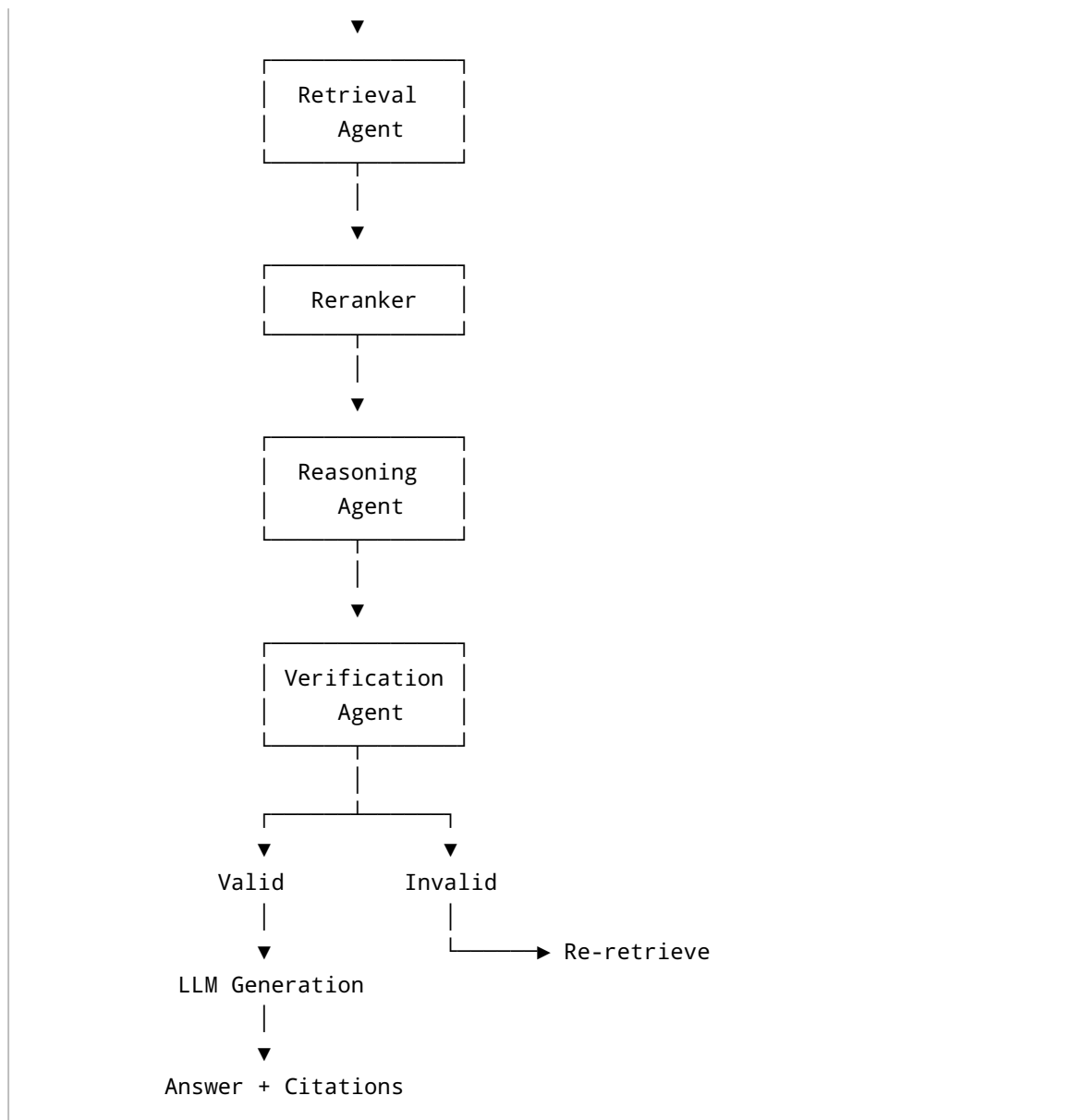
The core intelligence of DocMind is implemented using **LangGraph**.

Instead of a fixed:



workflow, the system uses specialised stages.





19. Query Planner Agent

The Query Planner interprets the user's request and determines how the system should approach it.

It identifies:

- query intent;
- relevant information;
- required modalities;
- required documents;
- whether multiple retrieval operations are necessary;
- whether comparison or multi-step reasoning is required.

For example:

"Compare the safety requirements in the 2024 and 2025 manuals."

The planner can classify this as a multi-document comparison task and request retrieval from both versions.

20. Retrieval Agent

The Retrieval Agent executes the retrieval strategy selected by the planner.

It can perform:

- semantic search;
- keyword search;
- hybrid search;
- metadata filtering;
- visual retrieval;
- table retrieval;
- multi-document retrieval.

If the retrieved information is insufficient, the workflow can perform another retrieval iteration.

21. Reasoning Agent

The Reasoning Agent analyses the retrieved context.

Its responsibilities include:

- synthesising retrieved information;
- comparing multiple documents;
- identifying relationships;
- resolving relevant evidence;
- performing multi-step reasoning;
- identifying possible contradictions.

The agent is instructed to reason from retrieved context rather than relying solely on the language model's internal knowledge.

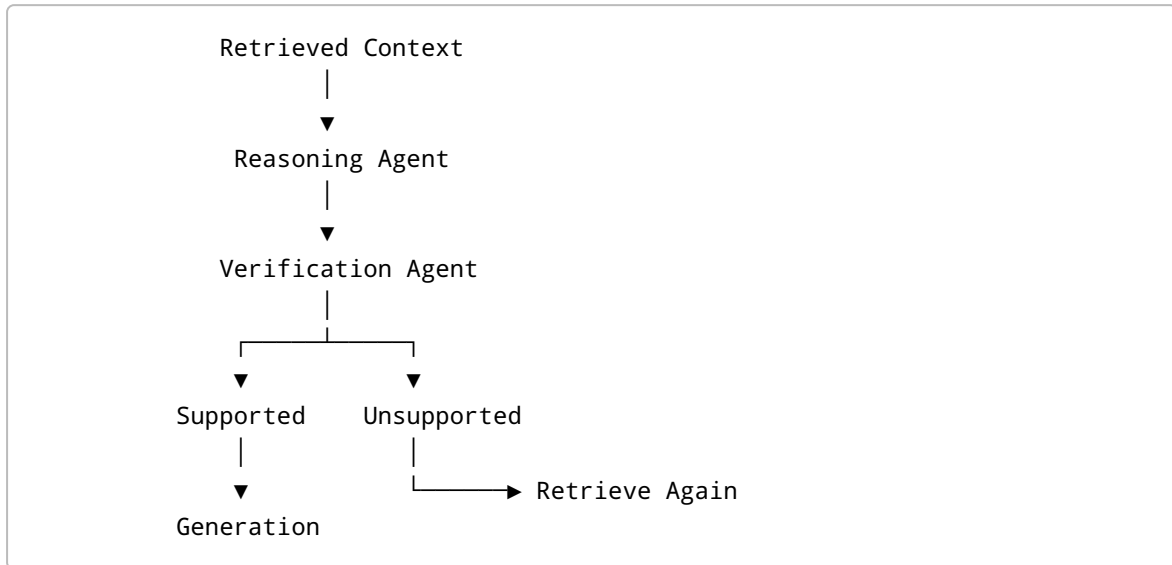
22. Verification Agent

The Verification Agent validates the proposed answer before final generation.

It checks:

- whether claims are supported by retrieved information;
- whether the retrieved context is relevant;

- whether citations are available;
- whether unsupported claims have been introduced;
- whether additional retrieval is necessary.



23. LLM Fallback Architecture

The final response generation and reasoning layer uses the following ordered model fallback chain:

1. GPT-OSS 120B

Primary model

Used as the first-choice model for reasoning, synthesis, agent decisions, and final response generation.

↓

2. Nemotron 3 Nano 30B-A3B

Fallback 1

Used when the primary model is unavailable or fails to complete the required operation.

↓

3. DeepSeek V4-Flash

Fallback 2

Provides an additional long-context reasoning and generation option.

↓

4. Qwen 27B

Fallback 3

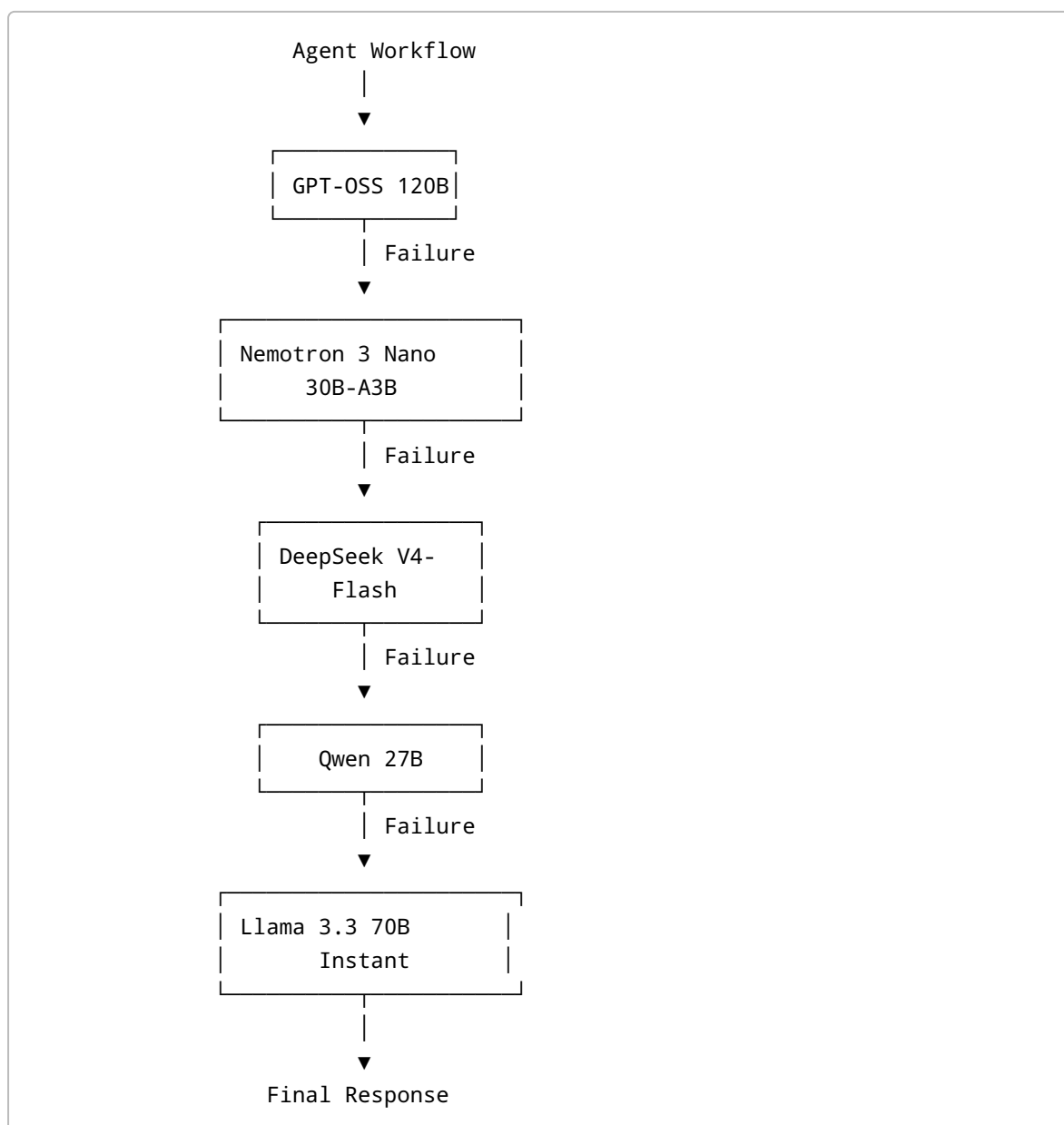
Provides another general-purpose reasoning and generation path.

↓

5. Llama 3.3 70B Instant

Final fallback

Used as the last available language-model option when the preceding models cannot fulfil the request.

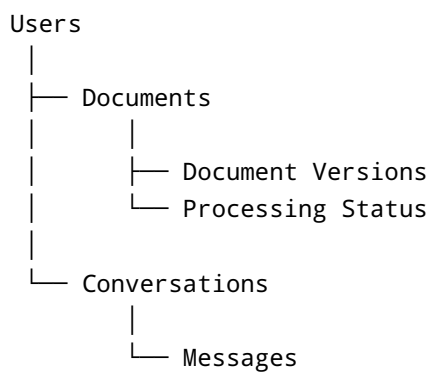


Important architectural distinction: Qwen2.5-VL is the dedicated **vision-language model** and is not part of this LLM fallback sequence. The fallback chain is responsible for general reasoning, synthesis, and response generation.

24. PostgreSQL Architecture

PostgreSQL stores structured application information that should not be managed exclusively through the vector database.

Potential entities include:



PostgreSQL can therefore manage:

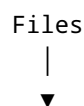
- user information;
- document records;
- document versions;
- processing status;
- conversations;
- messages;
- permissions;
- application metadata.

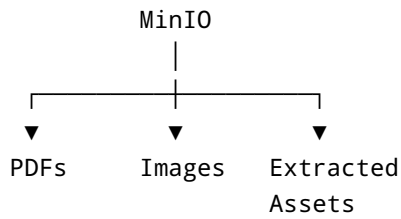
Qdrant remains responsible for vector retrieval.

25. Object Storage — MinIO

Original documents and extracted media are stored separately from the databases.

MinIO provides S3-compatible object storage.





PostgreSQL stores references to the stored objects, while Qdrant stores the searchable representations.

26. Redis

Redis provides fast temporary storage and caching.

Possible uses include:

- caching frequent queries;
- temporary agent state;
- processing status;
- rate limiting;
- session-related information;
- background task coordination.

Redis does not replace PostgreSQL or Qdrant.

27. Frontend Architecture

The frontend will use **React** with **Framer Motion**.

React

React will be used for:

- component-based interface development;
- document upload;
- dashboard;
- conversational UI;
- citation display;
- document management.

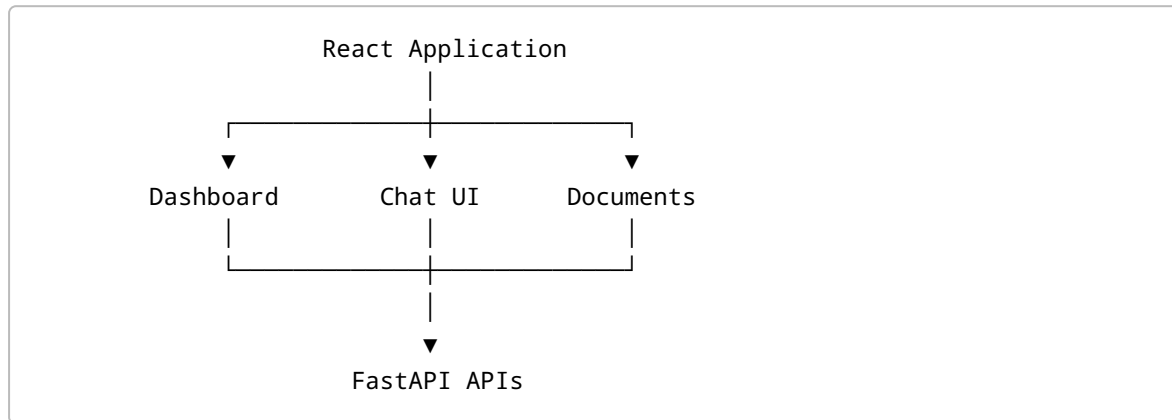
Framer Motion

Framer Motion will be used selectively for:

- page transitions;
- chat message animations;
- upload interactions;

- loading states;
- expandable citation elements;
- smooth UI transitions.

The interface will intentionally remain simple rather than exposing the complexity of the underlying agent architecture.



28. Proposed User Interface

The main interface will focus on three areas:

Dashboard

Provides:

- uploaded documents;
- document status;
- recent conversations;
- basic system information.

Chat Interface

Allows users to:

- ask natural-language questions;
- receive generated answers;
- continue conversations;
- view lightweight source citations.

Document Management

Allows users to:

- upload documents;
- view processing status;
- manage available documents;
- remove documents where authorised.

The interface will avoid unnecessary complexity and will not expose the internal multi-agent workflow to the user.

29. Citation System

Rather than presenting a separate evidence panel, DocMind will provide **lightweight inline or attached citations** with generated responses.

For example:

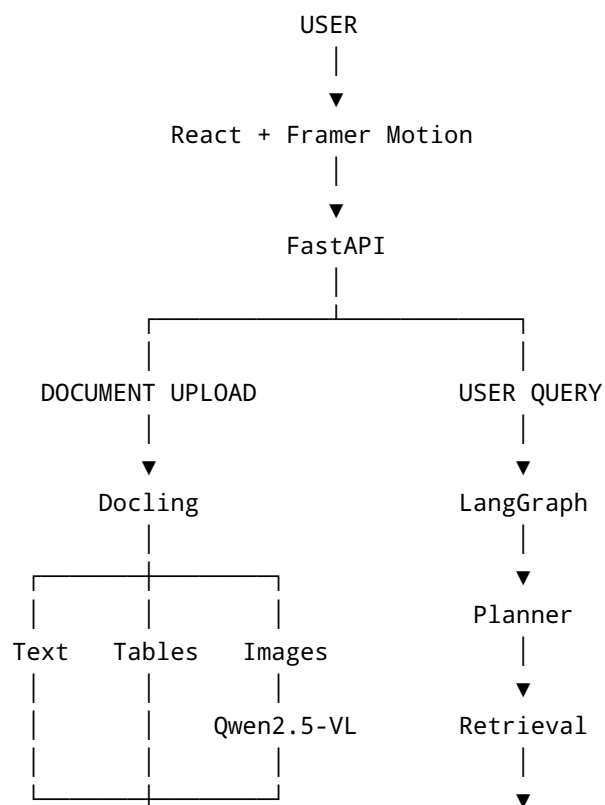
The recommended replacement interval is 500 operating hours.
[Maintenance Manual, Page 42]

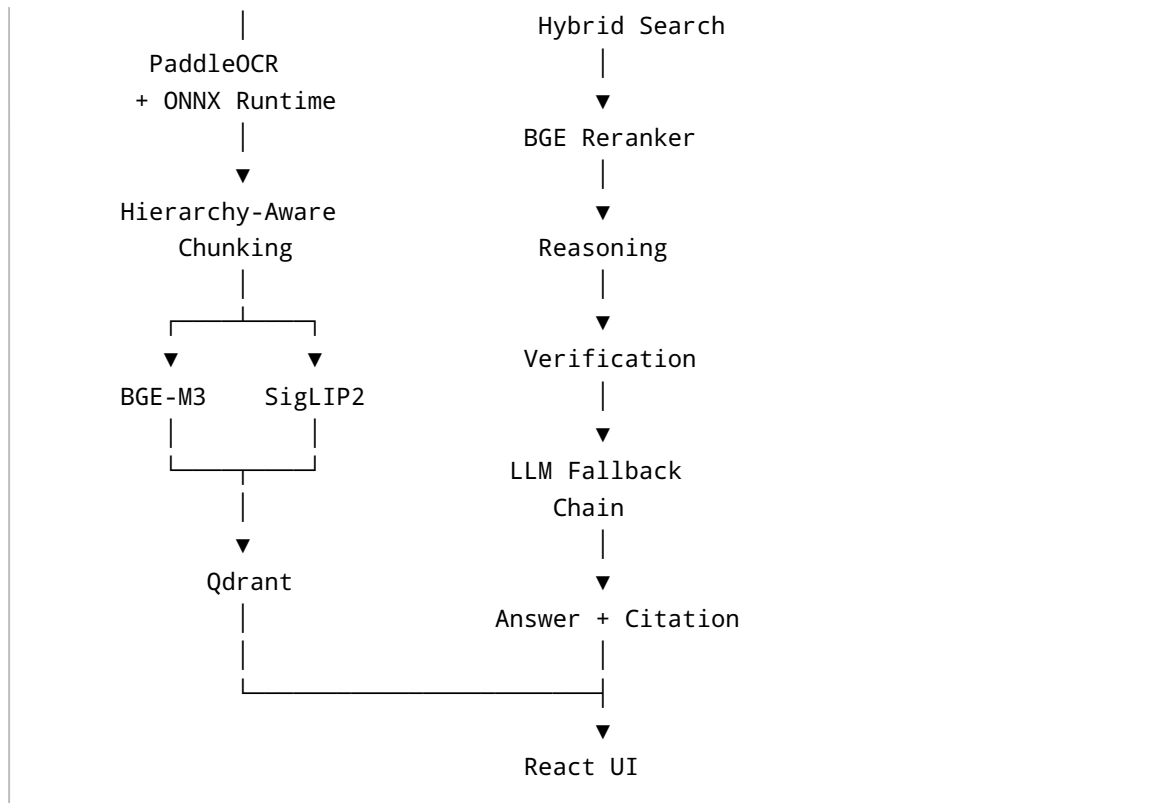
The 2025 revision additionally requires a pressure inspection.
[Maintenance Manual 2025, Page 47]

Clicking a citation can optionally open the relevant source document or page.

This keeps the interface simple while preserving answer transparency.

30. Complete End-to-End Flow





31. Example Use Case

User Query

"According to the 2024 and 2025 maintenance manuals, what are the steps for replacing the hydraulic filter and what changed between the two versions?"

Step 1 — Query Planning

The Planner identifies this as a multi-document comparison query.

Step 2 — Retrieval

Relevant sections from both manuals are retrieved.

Step 3 — Hybrid Search

Semantic and keyword retrieval identify potentially relevant passages.

Step 4 — Reranking

The BGE Reranker prioritises the most relevant results.

Step 5 — Visual Retrieval

If the procedure contains a relevant diagram, the associated visual content can be retrieved and interpreted by Qwen2.5-VL.

Step 6 — Reasoning

The Reasoning Agent compares the retrieved information.

Step 7 — Verification

The Verification Agent checks that the answer is supported by the retrieved sources.

Step 8 — Generation

GPT-OSS 120B generates the final response. If unavailable, the system moves through the configured fallback chain.

Example Response

The hydraulic filter replacement procedure consists of:

1. Shut down the hydraulic system.
2. Release system pressure.
3. Remove the existing filter.
4. Install the replacement filter.
5. Inspect the system for leakage.

The 2025 manual introduces an additional inspection requirement and modifies the recommended maintenance interval.

[2024 Maintenance Manual, Page 42]

[2025 Maintenance Manual, Page 47]

32. Implementation Plan

Phase 1 — Architecture and Infrastructure

- Finalise system architecture.
- Configure Git repository.
- Set up Docker environment.
- Configure FastAPI.
- Configure React frontend.
- Set up PostgreSQL, Qdrant, Redis, and MinIO.

Phase 2 — Document Processing

- Integrate Docling.
- Configure ONNX Runtime.
- Integrate PaddleOCR.
- Extract document structure.
- Extract tables and figures.
- Store original documents and extracted assets.

Phase 3 — Indexing

- Implement hierarchy-aware chunking.
- Generate BGE-M3 embeddings.
- Generate SigLIP2 visual embeddings.
- Store vectors in Qdrant.
- Store application metadata in PostgreSQL.

Phase 4 — Retrieval

- Implement semantic retrieval.
- Implement keyword/BM25 retrieval.
- Implement hybrid result fusion.
- Integrate BGE Reranker.
- Implement metadata filtering.

Phase 5 — Agentic Workflow

- Implement LangGraph state management.
- Build Query Planner Agent.
- Build Retrieval Agent.
- Build Reasoning Agent.
- Build Verification Agent.
- Integrate the LLM fallback chain.

Phase 6 — Multimodal Intelligence

- Integrate Qwen2.5-VL.
- Implement visual retrieval.
- Connect visual content with document metadata.
- Support image, diagram, and table-related queries.

Phase 7 — Frontend

- Build React dashboard.
- Build document upload interface.
- Build conversational interface.
- Implement citation display.
- Add Framer Motion interactions.
- Implement document management.

Phase 8 — Testing and Evaluation

- Build evaluation dataset.
 - Test retrieval quality.
 - Evaluate answer faithfulness.
 - Evaluate citation correctness.
 - Measure latency.
 - Compare baseline RAG with the proposed agentic architecture.
-

33. Evaluation Strategy

The project will be evaluated quantitatively rather than only through demonstration.

Retrieval Metrics

- Precision@K
- Recall@K
- Mean Reciprocal Rank
- Context Precision
- Context Recall

Generation Metrics

- Faithfulness
- Answer Relevance
- Citation Correctness
- Hallucination Rate

System Metrics

- End-to-end response latency;
- document processing time;
- retrieval latency;
- throughput;
- resource consumption.

RAGAS and **DeepEval** can be used to automate evaluation of RAG performance.

34. Baseline Comparison

A major component of the project evaluation will be comparing increasingly sophisticated retrieval architectures.

Baseline 1 — Basic RAG

Query → Vector Search → LLM → Answer

Baseline 2 — Enhanced RAG

Query → Hybrid Search → Reranker → LLM → Answer

Proposed DocMind

```
Query
  ↓
Planner Agent
  ↓
Adaptive Hybrid Retrieval
  ↓
Reranking
  ↓
Reasoning Agent
  ↓
Verification Agent
  ↓
LLM Fallback Chain
  ↓
Cited Answer
```

This comparison allows the project to measure whether agentic orchestration produces improvements over conventional RAG.

35. Security Considerations

Since enterprise documents may contain sensitive information, the system should consider:

- authenticated access;
- document-level permissions;
- secure file storage;
- controlled retrieval;
- API validation;
- protection against prompt injection;
- access logging;
- audit trails.

The retrieval layer must respect document permissions so that users cannot obtain information from documents they are not authorised to access.

36. Expected Outcomes

The completed system is expected to provide:

- structured document ingestion;
- OCR-based processing of scanned documents;
- multimodal document understanding;
- semantic and hybrid retrieval;
- visual content understanding;
- query planning through AI agents;
- multi-document reasoning;
- answer verification;
- citation-grounded responses;
- robust LLM fallback;
- an interactive web interface.

The project will demonstrate how modern Generative AI, multimodal models, vector retrieval, and agentic workflows can transform large document repositories into intelligent enterprise knowledge systems.

37. Future Scope

Future versions can incorporate:

1. Graph RAG and Knowledge Graph integration.
 2. Long-term conversational memory.
 3. Voice-based document interaction.
 4. Multilingual document understanding.
 5. Autonomous report generation.
 6. Enterprise platform integrations.
 7. Human-in-the-loop verification.
 8. Multi-tenant enterprise deployment.
 9. Continuous evaluation and model optimisation.
 10. Autonomous research and investigation agents.
-

38. Team Responsibilities

Team Leader

Responsible for overall system architecture, project planning, LangGraph agentic workflow, integration of major components, task coordination, testing, documentation, and final project delivery.

Member 2

Responsible for backend development, Docling-based document processing, OCR pipeline, PostgreSQL/Qdrant integration, APIs, and retrieval infrastructure.

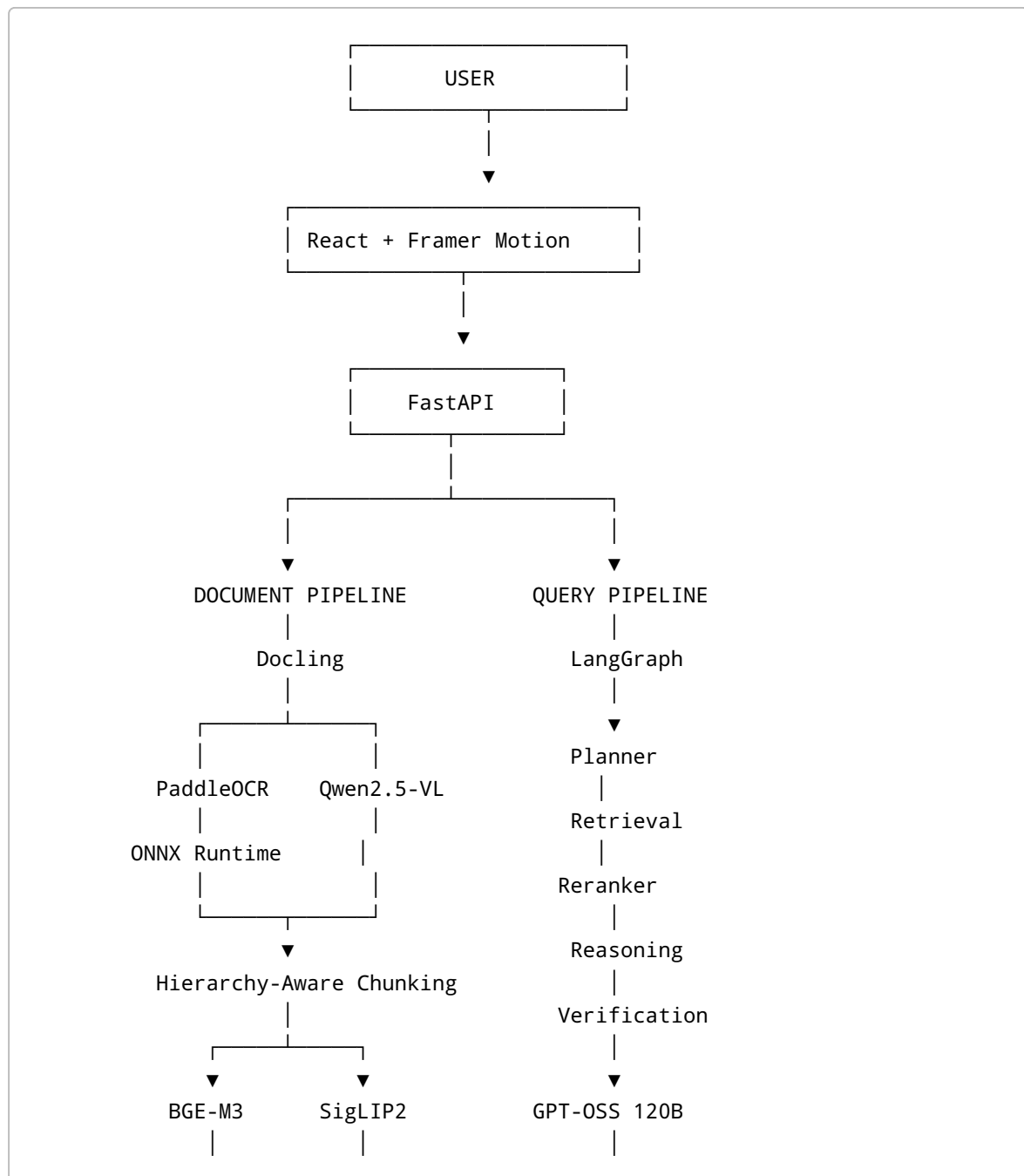
Member 3

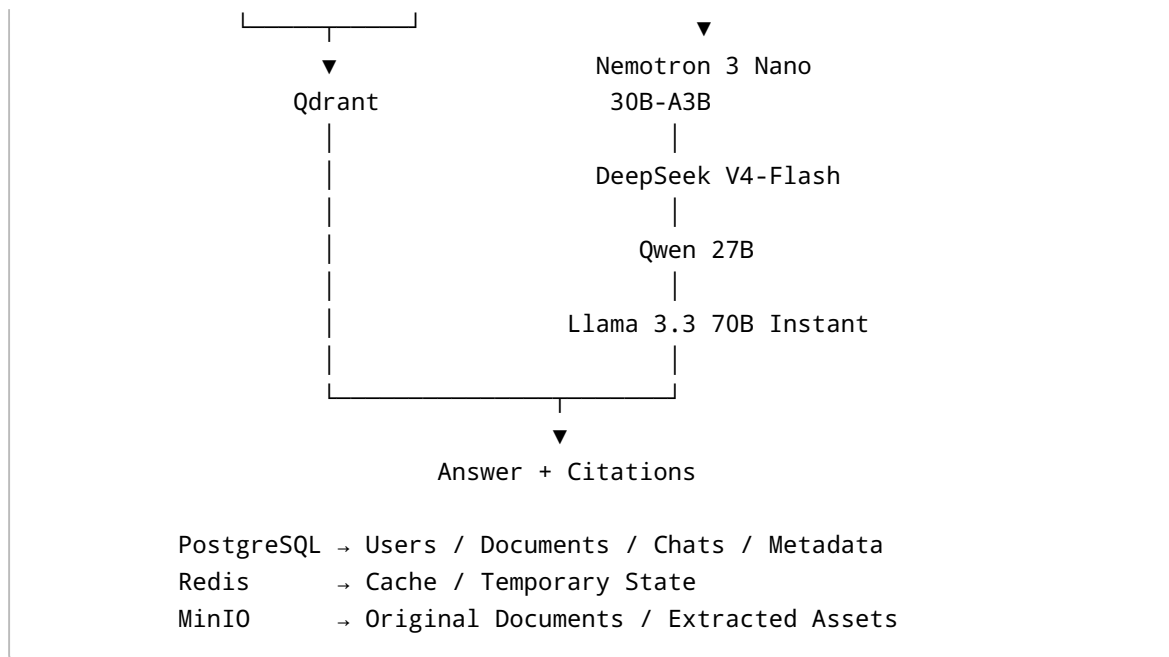
Responsible for React frontend development, Framer Motion interactions, dashboard, chat interface, document management, and citation presentation.

Member 4

Responsible for multimodal model integration, Qwen2.5-VL, embedding and reranking experiments, evaluation, testing, and performance analysis.

39. Final Technology Architecture





40. Conclusion

DocMind proposes a comprehensive enterprise document intelligence platform combining **structured document parsing, OCR, multimodal understanding, semantic retrieval, hybrid search, reranking, agentic orchestration, verification, and grounded language generation**.

The system uses **Docling** as the document parsing layer, **PaddleOCR through ONNX Runtime** for scanned content, and **Qwen2.5-VL** for visual understanding. **BGE-M3 and SigLIP2** provide textual and visual representations, while **Qdrant** enables efficient multimodal retrieval.

The **LangGraph-based agentic workflow** introduces query planning, adaptive retrieval, reasoning, and verification. The response generation layer uses a resilient model hierarchy:

GPT-OSS 120B → Nemotron 3 Nano 30B-A3B → DeepSeek V4-Flash → Qwen 27B → Llama 3.3 70B Instant.

PostgreSQL manages structured application data, Redis supports temporary state and caching, and MinIO provides object storage for original documents and extracted assets. The frontend uses **React and Framer Motion** to provide a clean and responsive user experience without exposing unnecessary internal system complexity.

Overall, DocMind aims to transform large collections of static enterprise documents into an **intelligent, multimodal, scalable, and explainable knowledge system**, while providing a strong technical foundation for evaluating the benefits of agentic RAG over conventional document question-answering systems.